```
 1  using System;
 2  using System.Collections.Generic;
 3  using System.Linq;
 4  using System.Text;
 5  using System.Threading.Tasks;
 6  using CustomProjectRPG.Entities;
 7  using CustomProjectRPG.Interfaces;
 8  using CustomProjectRPG.Manager;
 9  using CustomProjectRPG.Core.Utilities;
10
11
12
13  namespace CustomProjectRPG.World
14  {
15      public class Area
16      {
17          private readonly string _name;
18          private readonly Vector2D _size; // Area dimensions (width,
              height)
19          private readonly List<Entity> _entities;
20          private readonly List<ICollidable> _staticAssets;
21          private readonly Dictionary<string, Area> _subAreas;
22          private readonly CollisionManager _collisionManager;
23          private bool _isLocked;
24          private string _unlockCondition; // E.g., knowledge clue ID
25
26          public string Name => _name;
27          public Vector2D Size => _size;
28          public IReadOnlyList<Entity> Entities => _entities.AsReadOnly();
29          public IReadOnlyList<ICollidable> StaticAssets =>
              _staticAssets.AsReadOnly();
30          public IReadOnlyDictionary<string, Area> SubAreas =>
              _subAreas.AsReadOnly();
31          public bool IsLocked => _isLocked;
32
33          public Area(string name, Vector2D size, CollisionManager
              collisionManager)
34          {
35              _name = name;
36              _size = size;
37              _entities = new List<Entity>();
38              _staticAssets = new List<ICollidable>();
39              _subAreas = new Dictionary<string, Area>();
40              _collisionManager = collisionManager;
41              _isLocked = false;
42              _unlockCondition = null;
43          }
44
45          public Area(string name, Vector2D size, CollisionManager
```

```
            collisionManager, string unlockCondition)
46              : this(name, size, collisionManager)
47          {
48              _isLocked = true;
49              _unlockCondition = unlockCondition;
50          }
51
52          public void AddEntity(Entity entity)
53          {
54              if (entity == null)
55              {
56                  throw new System.ArgumentNullException(nameof(entity));
57              }
58              if (!_entities.Contains(entity))
59              {
60                  _entities.Add(entity);
61                  _collisionManager.AddCollidable(entity);
62              }
63          }
64
65          public void RemoveEntity(Entity entity)
66          {
67              if (entity == null)
68              {
69                  throw new System.ArgumentNullException(nameof(entity));
70              }
71              if (_entities.Remove(entity))
72              {
73                  _collisionManager.RemoveCollidable(entity);
74              }
75          }
76
77          public void AddStaticAsset(ICollidable asset)
78          {
79              if (asset == null)
80              {
81                  throw new System.ArgumentNullException(nameof(asset));
82              }
83              if (!_staticAssets.Contains(asset))
84              {
85                  _staticAssets.Add(asset);
86                  _collisionManager.AddCollidable(asset);
87              }
88          }
89
90          public void RemoveStaticAsset(ICollidable asset)
91          {
92              if (asset == null)
93              {
```

```
94                  throw new System.ArgumentNullException(nameof(asset));
95              }
96              if (_staticAssets.Remove(asset))
97              {
98                  _collisionManager.RemoveCollidable(asset);
99              }
100         }
101
102         public void AddSubArea(string subAreaName, Area subArea)
103         {
104             if (subArea == null)
105             {
106                 throw new System.ArgumentNullException(nameof(subArea));
107             }
108             if (!_subAreas.ContainsKey(subAreaName))
109             {
110                 _subAreas.Add(subAreaName, subArea);
111             }
112         }
113
114         public void RemoveSubArea(string subAreaName)
115         {
116             _subAreas.Remove(subAreaName);
117         }
118
119         public bool TryUnlock(string knowledgeClue)
120         {
121             if (_isLocked && _unlockCondition == knowledgeClue)
122             {
123                 _isLocked = false;
124                 return true;
125             }
126             return false;
127         }
128
129         public void Update()
130         {
131             foreach (var entity in _entities)
132             {
133                 entity.Update();
134             }
135             _collisionManager.CheckCollisions();
136         }
137
138         public void Draw()
139         {
140             foreach (var entity in _entities)
141             {
142                 entity.Draw();
```

```
143             }
144             // Draw static assets (placeholder for SplashKit rendering)
145             foreach (var asset in _staticAssets)
146             {
147                 if (asset is StaticAsset staticAsset)
148                 {
149                     staticAsset.Draw();
150                 }
151             }
152         }
153 
154         public bool IsWithinBounds(Vector2D position)
155         {
156             return position.X >= 0 && position.X < _size.X && position.Y
                  >= 0 && position.Y < _size.Y;
157         }
158     }
159 }
```